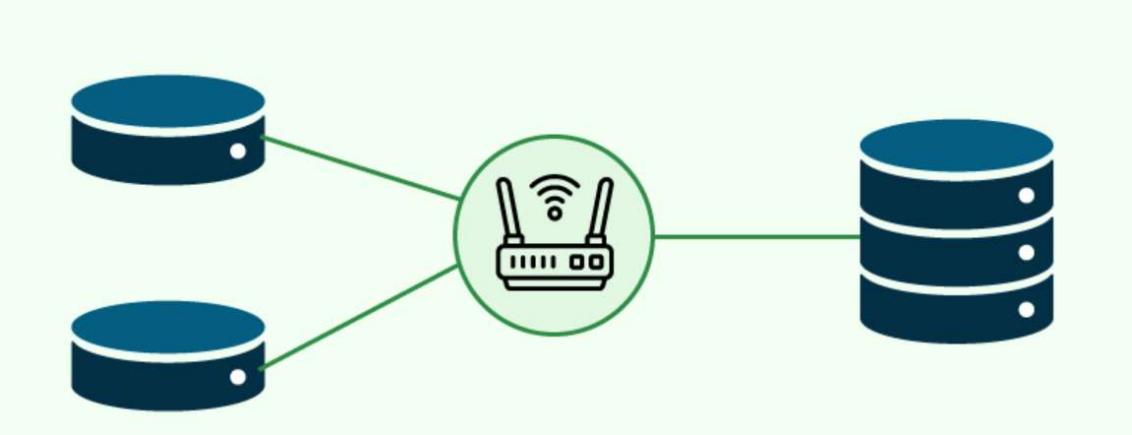
# Unit 4

# Middleware for IoT

## Middleware for IoT

Middleware are software tools that act as intermediaries between different applications, systems, or services, facilitating their communication and interaction. They ensure that data and requests can be exchanged smoothly and efficiently, even if the systems involved are built using different technologies.

They handle various tasks such as data translation, message queuing, authentication, and connectivity, making it easier to integrate and manage complex software environments. The middleware serves as the link between users, data, and applications.



## purpose of Middleware

There are several uses for middleware. It first controls access to different back-end resources. A connection pool could be established by a middleware component to enable quick and easy access to a well-liked back-end database.

It is also capable of establishing links to topics and message queues. Moreover, middleware software may control access to cloud-based services such as Amazon Simple Storage Service.

Middleware software is capable of executing logic according to the client's request.

In load balancing, transaction management, and concurrent processing, middleware is crucial. Typically, middleware software can grow both vertically and horizontally to assist in distributing incoming client requests over several servers, virtual machines, or cloud availability zones.

Securing access to back-end resources is mostly dependent on middleware

The capacity of middleware software to challenge clients is contingent upon two requirements: a secure connection, established through technologies such as SSL, and authentication, accomplished through the use of a digital certificate or a username and password combination.

## Categories of Middleware

Platform middleware: It facilitates the development and delivery of software by giving application program logic a runtime hosting environment, like a container. Web servers, application servers, and content management systems (CMSs) are examples of tools that facilitate application development and delivery and are included in platform middleware.

Enterprise application integration middleware: Programmers can develop business apps with the help of enterprise application integration middleware, which eliminates the need to design unique connectors for every application. By offering a layer of functionality for data integrity and B2B connection, middleware in this case facilitates software collaboration.

**Types of Middleware**

Remote Procedure Call (RPC): A program can ask for a service from another program running on a different computer or network using this protocol, which is provided via middleware.

Messaging middleware: It makes it easier for dispersed apps and services to communicate with one another.

Embedded middleware: enables integration and communication between real-time operating systems and embedded applications.

API middleware: lets programmers design and control the APIs in their applications.

Asynchronous data streaming middleware : By duplicating data streams in an intermediary repository, asynchronous data streaming middleware facilitates data exchange between apps.

Transaction or transactional middleware: This uses transaction process monitoring to make sure transactions proceed smoothly from one stage to the next.

**How Does Middleware Work?**

In its most basic form, middleware allows developers to construct applications without the need to design a unique integration each time they want to link to devices, data sources, computational resources, or services.

This is achieved by offering services that facilitate communication between various applications and services via standard messaging frameworks, including web services, XML (extensible markup language), REST (representational state transfer), JSON (JavaScript object notation), and SOAP (simple object access protocol). In most cases.

**Incoming Request**

A client sends a request (e.g., GET /users).

Express routes the request through the stack of middleware.

**Middleware Function**

A middleware is just a function with the signature:

function (req, res, next) { ... }

req: the request object (data from client).

res: the response object (data to send back).

next: a callback function that moves control to the next middleware.

**Embedded IoT Devices**

Introduction to IoT and embedded systems

The Internet of Things (IoT) and embedded systems are two interconnected technologies that have revolutionised the way we live and work. IoT refers to a network of physical devices embedded with sensors, software, and connectivity, enabling them to collect and exchange data.

Embedded systems, on the other hand, are specialised operating systems designed to perform dedicated functions within larger systems. Together, they form the backbone of modern smart technologies, driving innovation across various industries.

**What is IoT?**

IoT, or the Internet of Things, is a vast network of physical devices, vehicles, home appliances, and other items embedded with sensors, software, and connectivity. This network allows these devices to collect and exchange data with other devices and systems over the internet.

IoT devices can range from simple sensors that monitor environmental conditions to complex systems that manage entire industrial processes. They are used in a wide array of industries, including healthcare, manufacturing, transportation, and smart homes, transforming how we interact with the world around us.

**What is an embedded system?**

An embedded system devices are small computer systems designed to perform a specific task or set of tasks. It typically consists of a combination of hardware and software components that work together to provide a particular function or service.

Embedded systems are ubiquitous, found in consumer electronics like smartphones and smartwatches, industrial control systems, medical devices, and automotive systems. These systems are integral to the operation of many modern technologies, providing the necessary control and processing capabilities.

**Characteristics of IoT embedded systems**

The difference between IoT and embedded system in one word is connectivity. Even a pacemaker is an embedded system. By letting that device communicate to the outside world, such as transmitting heart-rate data to a central database, it turns out into an IoT embedded system.

The other difference between an embedded system and IoT is that IoT refers more to a class of devices that represent the newly connected world. But an embedded system refers very specifically to the hardware used in these devices.

The primary difference between an embedded operating system and IoT is that the IoT operating system needs to support connectivity. The capability of IoT networking of embedded system devices is what differentiates plain embedded systems from IoT embedded systems.

Every IoT use case will have an embedded system as part of it, which is why the term for it is IoT embedded system. First, there is an embedded system, and then come the features that turn it into an IoT device.

Embedded systems can be highly sophisticated in themselves, such as robotic systems used in manufacturing or data warehouses. But until that device can communicate and has connectivity with other IoT embedded systems, its function as a pure embedded system remains limited to it's single intended purpose. The distinction between embedded system and IoT embedded system is only one of its ability to communicate.

**What's the difference between IoT and embedded systems?**

Traditionally, embedded systems are a combination of hardware and software that perform a dedicated function and typically control the operations of a machine it's embedded within. The hardware elements include microcontrollers or microprocessors, actuators and sensors. The software components include an operating system, device drivers, and application software.

The Internet revolutionised the way we interact with technology. Today, embedded systems also include communication models, like wi-fi, Bluetooth or cellular connectivity, which allow them to collect and exchange data over the Internet that can be used later for monitoring and various analyses. **Embedded systems that are used in IoT devices are called IoT embedded systems.**

IoT embedded systems possess several distinctive characteristics that set them apart from traditional embedded systems:

- **Connectivity**: IoT embedded systems are designed to connect to the internet or other networks, enabling communication with other devices and systems. This connectivity is crucial for data exchange and remote monitoring.
- **Sensing and actuation**: These systems often include sensors that detect changes in the physical environment and actuators that perform actions based on sensor data. This capability allows IoT devices to interact dynamically with their surroundings.
- **Data processing**: IoT embedded systems can process data in real-time, making decisions and taking actions based on the data they collect. This real-time processing is essential for applications that require immediate responses.
- **Energy efficiency**: Given that many IoT devices operate on batteries or other limited power sources, energy efficiency is a critical design consideration. IoT embedded systems are optimised to consume minimal power while maintaining functionality.

**How do IoT embedded systems work?**

IoT embedded systems enable devices to become part of the Internet-of-Things ecosystem. They are used widely in smart homes, healthcare, automotive and especially in the Industry 4.0 sector.

The sensors, which are part of the embedded system, gather information about physical surroundings, for example, temperature, vibrations or humidity. The collected data is then exchanged with other devices or systems over the Internet. The data can then be analysed and

used to make more informed decisions, automate processes or provide real-time feedback on the state of a device.

IoT embedded systems bring many benefits to modern manufacturing. Let's take a look at a couple of the most common applications.



**Hardware and software for IoT embedded systems**

IoT embedded systems are composed of both hardware and software components, each playing a vital role in their operation.

- **Microcontrollers**: These small computers are designed to perform specific tasks, often controlling sensors and actuators within the IoT embedded system.
- **Sensors**: Devices that detect changes in the physical world, such as temperature, humidity, or motion, providing the data needed for the system to function.
- **Actuators**: Devices that take action based on sensor data, such as turning on a light or activating a motor, enabling the system to interact with its environment.
- **Communication modules**: Components that allow IoT embedded systems to communicate with other devices and systems over the internet, facilitating data exchange and remote control.
- **Power management**: Systems that manage the power consumption of the IoT embedded system, ensuring efficient use of batteries or other power sources.

**Types of IoT embedded systems**

IoT embedded systems come in many forms, but they all share one thing in common: software. Whether it's lightweight firmware or a full embedded operating system, the software is what enables these systems to communicate, connect, and perform their roles in a larger IoT ecosystem.

You'll often hear the term embedded device used alongside embedded system and while they're related, there's a difference worth noting. Embedded devices are typically single-purpose components designed to do one job extremely well. Think of them as highly specialised tools in a much bigger toolbox. That's why they're also called Single Purpose Devices or Dedicated Devices.

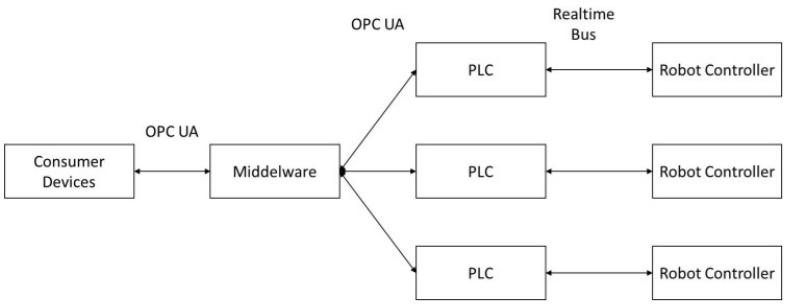
These devices are built with one task in mind, often tailored to a specific business use case. You'll find them everywhere from retail to healthcare, from automotive to consumer electronics.

Common examples include:

- Kiosk terminals
- Point-of-sale (POS) systems
- Self-checkout stations

Behind each one is an IoT embedded system doing the heavy lifting—reliable, efficient, and purpose-built.

**Communication middleware**




The main function of this architecture is to act as a **bridge between user-side applications (consumer devices)** and **industrial machines (PLCs and robot controllers)**. This is achieved using middleware and communication protocols like OPC UA.

**1. Consumer Devices**

- These include user interfaces such as computers, mobile apps, dashboards, or monitoring systems.

- They **request data** (e.g., machine status) or **send commands** (e.g., start/stop operations).

**2. Middleware**

- Middleware is the **central layer** in the diagram.
- It performs several important functions:
    - Data collection from industrial devices
    - Data processing and formatting
    - Protocol conversion
    - Security enforcement
- It ensures that consumer devices don't need to directly interact with complex industrial systems.

Communication between **consumer devices and middleware** uses **OPC UA**, which is:

- Platform-independent
- Secure
- Widely used in industrial IoT (IIoT)

**3. PLCs (Programmable Logic Controllers)**

- PLCs are industrial controllers that:
    - Monitor sensors
    - Control machines and processes
- Middleware communicates with multiple PLCs via OPC UA.
- Each PLC acts as a **local control unit**.

**4. Robot Controllers**

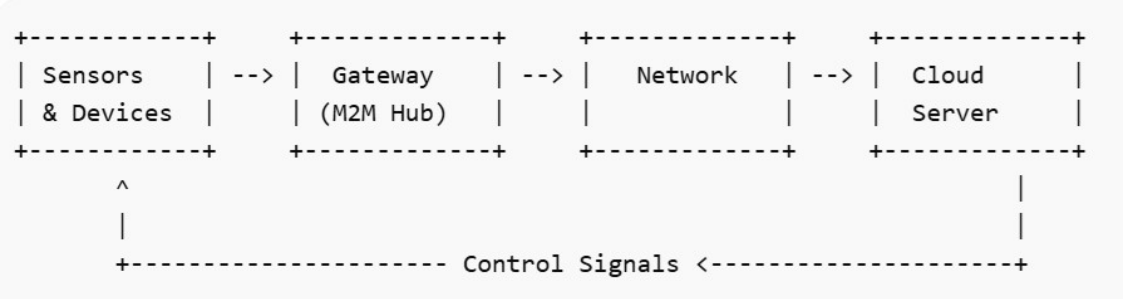
- These control robotic systems in manufacturing or automation.
- Each PLC is connected to a robot controller via a **Realtime Bus**.

**5. Realtime Bus**

- This is a **high-speed communication channel** between PLCs and robot controllers.
- It ensures:
    - Low latency

    - Deterministic (predictable) communication
- Critical for time-sensitive operations like robotics.

**M2M**




**1. Sensors & Devices**

- This is the **first layer (data source layer)**.
- Includes:
    - Sensors (temperature, pressure, humidity)
    - Actuators (motors, valves)
    - Embedded devices
- **Function:**
    - Collect real-time data from the environment or machines
    - Send raw data to the gateway

👉 Example: A temperature sensor measuring machine heat

**2. Gateway (M2M Hub)**

- Acts as a **bridge between devices and the network**.
- Also called an M2M hub or edge device.

**Functions:**

- Data aggregation (collects data from multiple sensors)
- Protocol conversion (e.g., Zigbee → IP)
- Data filtering and preprocessing
- Security (authentication, encryption)

👉 It reduces the load on the cloud by processing data locally.

**3. Network**

- Responsible for **transmitting data** from gateway to cloud.

**Types of networks:**

- Wired: Ethernet
- Wireless: Wi-Fi, Bluetooth, Cellular (4G/5G)

**Function:**

- Ensures reliable and continuous communication between devices and server

**4. Cloud Server**

- This is the **central processing unit** of the system.

**Functions:**

- Data storage (databases)
- Data analysis (analytics, AI/ML)
- Decision-making
- Device management

👉 Example: Detecting abnormal temperature and triggering an alert

**5. Control Signals (Feedback Path)**

- The bottom arrow in the diagram shows **reverse communication**.

**Flow:**
Cloud Server → Network → Gateway → Sensors/Devices

**Function:**

- Sends commands back to devices
- Enables automation and control

👉 Example:

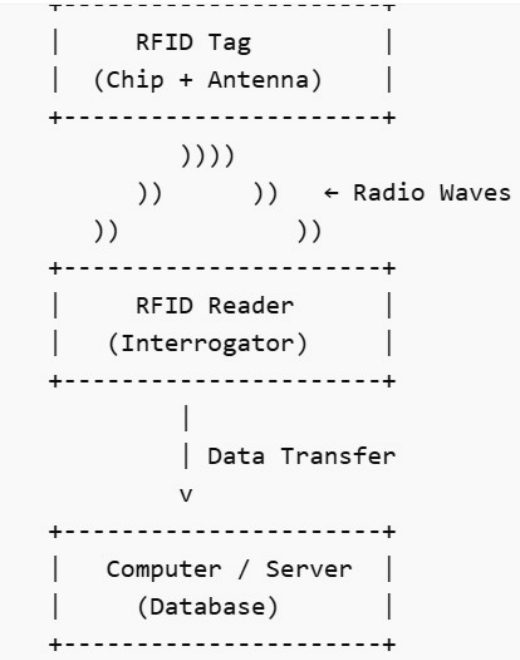
- If temperature is too high → cloud sends command → machine shuts down automatically

**RFID System Diagram**

**Functions of RFID System**

- **Identification:** Unique ID for each object
- **Tracking:** Real-time location monitoring

- **Automation:** No manual scanning required
- **Data Storage:** Stores product or asset information
- **Security:** Used in access control system




**1. RFID Tag (Transponder)**

- Attached to an object (product, card, asset)
- Contains:
    - **Microchip:** Stores unique ID/data
    - **Antenna:** Sends/receives signals

**Function:**

- Stores information about the object
- Transmits data when activated by reader

**2. RFID Reader (Interrogator)**

- A device that communicates with tags

**Function:**

- Sends **radio signals** to detect tags
- Activates passive tags (provides power)

- Receives data from tags
- Converts signals into digital data

**3. Antenna**

- Present in both tag and reader

**Function:**

- Enables wireless communication
- Determines reading range and signal strength

**4. Computer / Server (Database)**

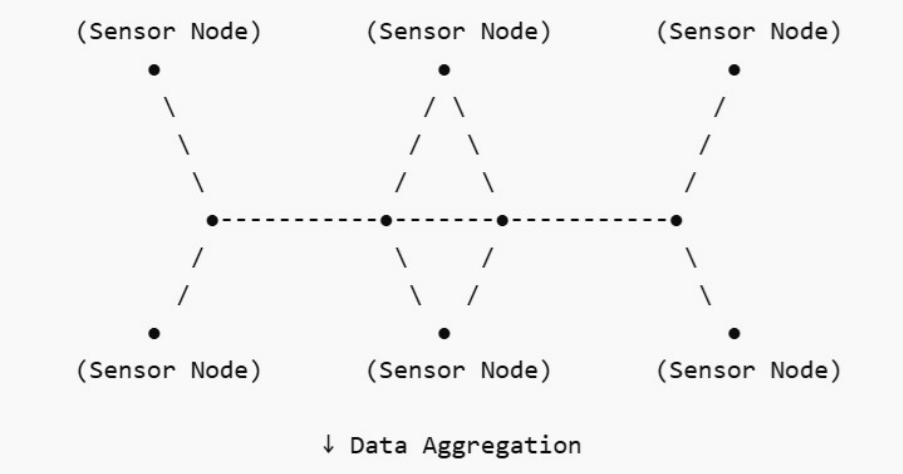
- Connected to RFID reader

**Function:**

- Stores received data
- Processes information
- Displays results to users
- Triggers actions (alerts, updates, etc.)

**Wireless Sensor Network (WSN)**

A Wireless Sensor Network (WSN) is a network of small sensor nodes that collect data from the environment and communicate wirelessly to a central system (sink/base station) for processing.




**1. Sensor Nodes**

- Small devices deployed in large numbers
- Consist of:
    - Sensor (temperature, humidity, etc.)
    - Microcontroller
    - Battery
    - Wireless transceiver

**Function:**

- Sense environmental data
- Transmit data to neighboring nodes or sink

**2. Sink / Base Station**

- Central node in the network

**Function:**

- Collects data from all sensor nodes
- Sends data to server/cloud
- Acts as gateway between WSN and external network

**3. Communication Network**

- Wireless links between nodes

**Function:**

- Enables multi-hop communication
- Data travels node-to-node until it reaches sink

**4. Server / Cloud**

- Stores and processes data

**Function:**

- Data analysis
- Decision-making
- Visualization

**5. User Application**

- Interface for users

**Function:**

- Displays data (graphs, alerts)
- Allows monitoring and control

**Applications of WSN**

- Environmental monitoring (temperature, pollution)
- Smart agriculture
- Military surveillance
- Healthcare monitoring
- Industrial automation

**Advantages**

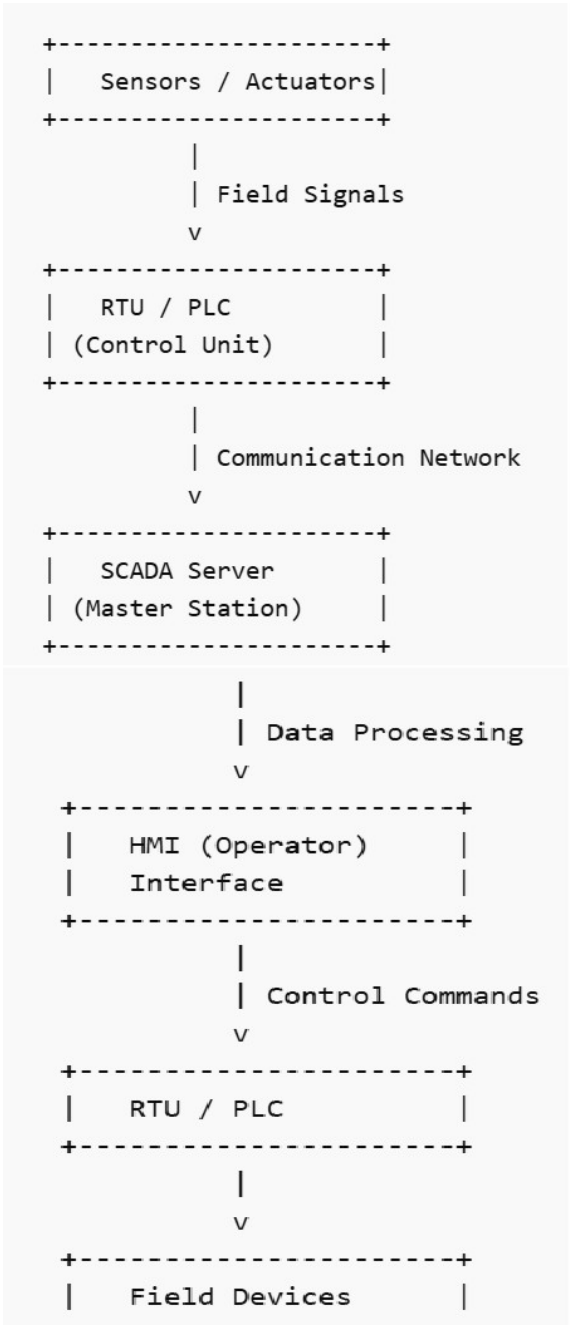
- Easy deployment
- No wiring required
- Covers large areas
- Low cost (small nodes)

**Limitations**

- Limited battery life
- Security issues
- Data loss in harsh environments
- Limited processing power

**SCADA**

SCADA (Supervisory Control and Data Acquisition) is a system used in industries to monitor, control, and analyze processes remotely in real time. It is widely used in power plants, water systems, manufacturing, and oil & gas.




**1. Sensors and Actuators**

- Sensors measure parameters (temperature, pressure, flow)
- Actuators perform actions (valves, motors)

**Function:**

- Interface with physical environment

**2. RTU / PLC (Remote Terminal Unit / Controller)**

- RTU: Used in remote locations
- PLC: Used for local control

**Function:**

- Collect data from sensors
- Execute control logic
- Send data to SCADA server

**3. Communication Network**

- Connects field devices to control center

**Types:**

- Wired (Ethernet, fiber)
- Wireless (radio, cellular)

**Function:**

- Transfers data and control signals

**4. SCADA Server (Master Station)**

- Central system

**Function:**

- Data acquisition
- Data storage (historian)
- Alarm generation
- Supervisory control

**5. HMI (Human Machine Interface)**

- Graphical interface for operators

**Function:**

- Displays real-time data
- Shows alarms and trends
- Allows manual control

**Applications of SCADA**

- Power generation and distribution
- Water treatment plants
- Oil and gas pipelines
- Manufacturing industries
- Smart grids

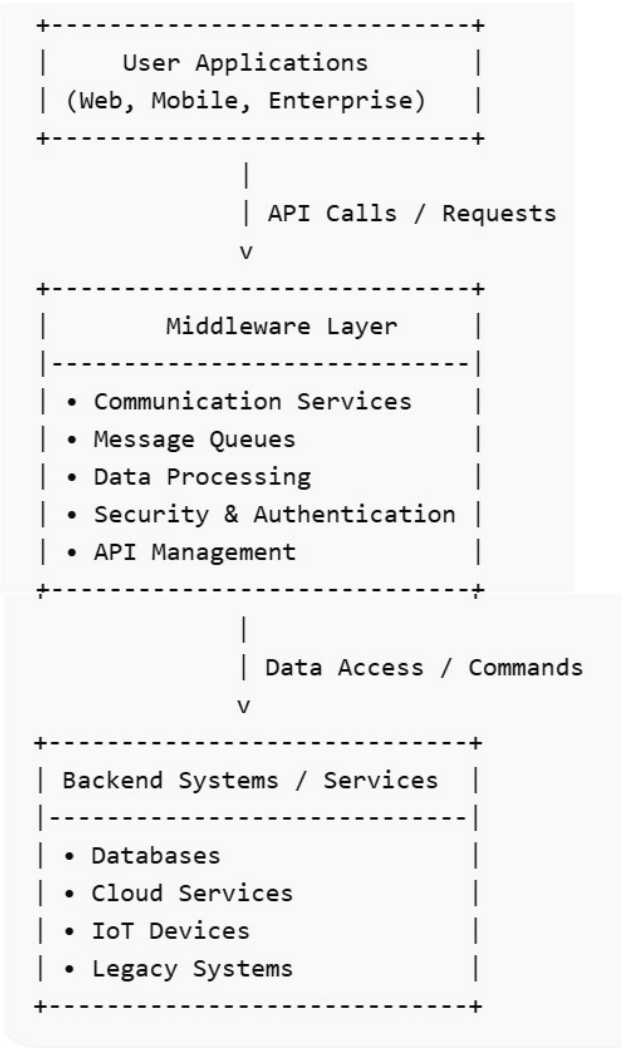
**Advantages**

- Centralized control
- Improved efficiency
- Faster decision-making
- Reduced manpower

**Limitations**

- High installation cost
- Security vulnerabilities
- Requires skilled operators

**Software middleware**




**1. Communication Services**

- Responsible for **data exchange between systems**
- Supports protocols like HTTP, TCP/IP, MQTT

👉 Function:

- Enables different applications and devices to communicate seamlessly
- Acts as the **core connectivity layer**

**2. Message Queues**

- Stores messages temporarily before delivery

👉 Function:

- Ensures **reliable communication**

- Handles asynchronous communication (sender and receiver need not be active at the same time)
- Prevents data loss during system overload

**3. Data Processing**

- Processes and transforms data

👉 Function:

- Converts data formats (JSON, XML, etc.)
- Filters and aggregates data
- Prepares data for backend systems

**4. Security & Authentication**

- Protects system and data

👉 Function:

- Verifies user identity (authentication)
- Controls access (authorization)
- Encrypts data for secure transmission

**5. API Management**

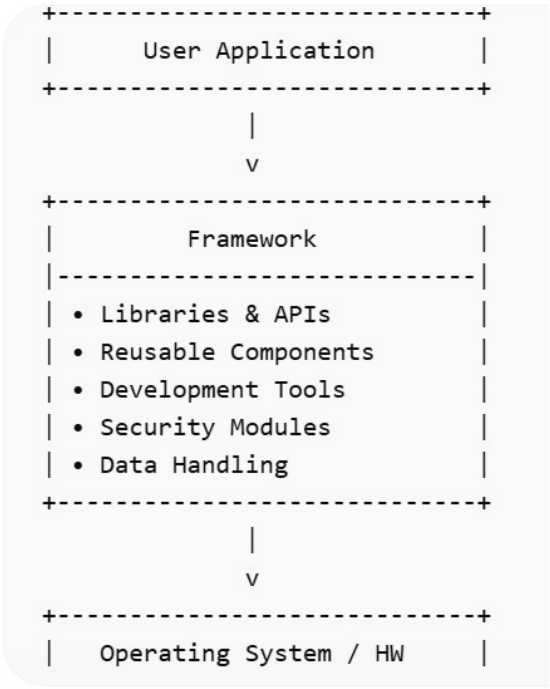
- Provides standardized interfaces (APIs)

👉 Function:

- Allows applications to access services easily
- Manages API requests and responses
- Controls API usage (rate limiting, monitoring)

**Framework**

A Framework is a pre-built software structure that provides a foundation for developing applications. It includes ready-made components, libraries, tools, and rules to simplify and speed up development.




**1. Web Frameworks**

- Used to build web applications
- Examples:
    - Django
    - Spring Framework

**2. Application Frameworks**

- Used for general software development
- Provide GUI, database, and logic support

**3. Mobile Frameworks**

- Used for mobile app development
- Example: Flutter

**4. IoT Frameworks**

- Used in IoT systems for device communication and data handling
- Example: Node-RED

**5. Testing Frameworks**

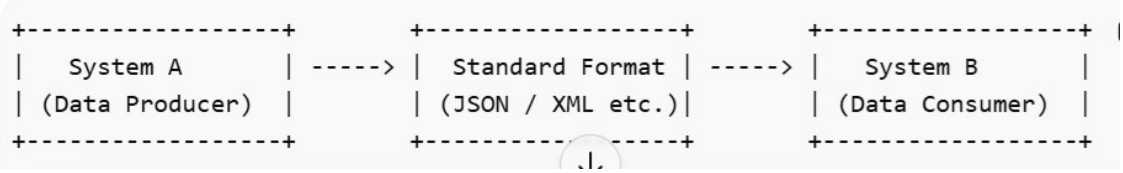
- Used for software testing
- Example: JUnit

**Data Standards**

Data Standards are rules and formats used to organize, represent, and exchange data between different systems so that they can understand each other correctly.

**Functions of Data Standards**

- **Interoperability:** Systems can work together
- **Consistency:** Same format everywhere
- **Accuracy:** Reduces errors
- **Data Sharing:** Easy exchange of information
- **Integration:** Connects multiple systems




**1. Data Format Standards**

- Define how data is structured

Examples:

- JSON
- XML
- CSV

**2. Communication Standards**

- Define how data is transmitted

Examples:

- HTTP
- MQTT
- CoAP

**3. Semantic Standards**

- Define meaning of data

Examples:

- Units (°C, kg, meter)
- Naming conventions

**4. Industry-Specific Standards**

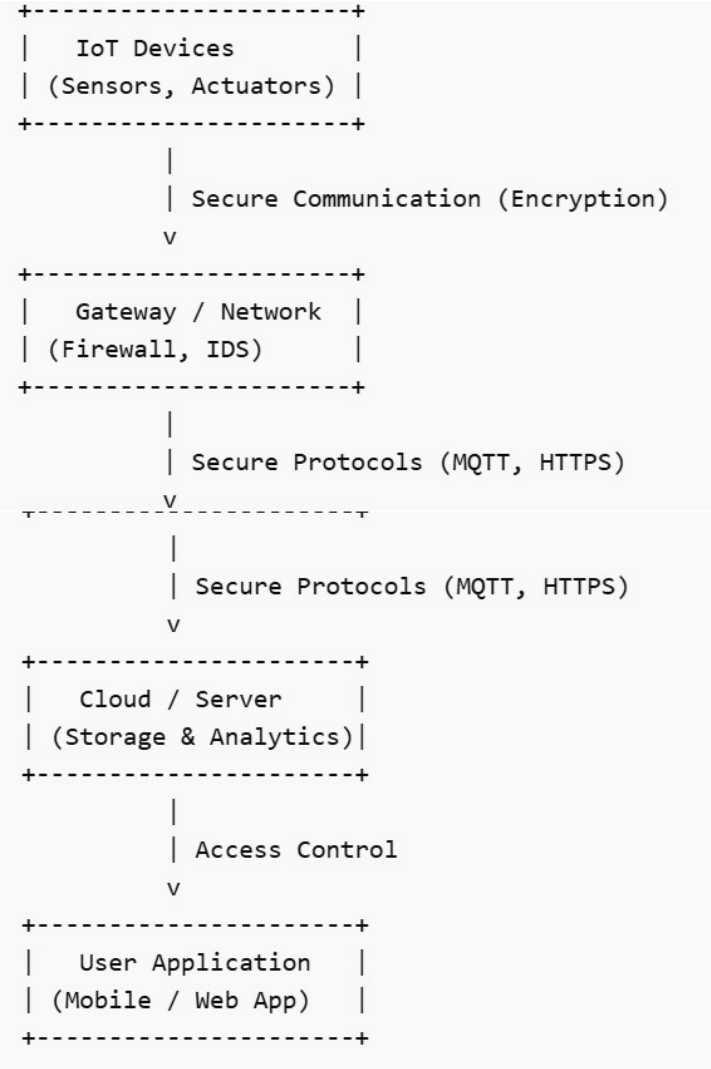
- Used in specific domains

Examples:

- OPC UA (Industrial IoT)
- HL7 (Healthcare)

**IoT Information Security**

IoT Information Security refers to protecting IoT devices, networks, and data from unauthorized access, attacks, and misuse. It ensures that data remains confidential, accurate, and available.




1. IoT Devices (Sensors & Actuators)

- These are the data sources
- Examples: temperature sensor, smart meter

Function:

- Collect real-time data
- Send data to gateway securely

2. Gateway / Network

- Acts as an intermediate layer

Function:

- Transfers data from devices to cloud
- Applies:

    - Firewall protection
    - Intrusion Detection Systems (IDS)
- Ensures secure communication

3. Secure Communication

- Data is protected during transmission

Function:

- Uses encryption techniques like
    - AES encryption
- Prevents hacking and data theft

4. Cloud / Server

- Central system for processing data

Function:

- Stores data securely
- Performs analytics
- Detects threats and anomalies

5. User Application

- Interface for users (mobile/web app)

Function:

- Displays data
- Allows control of devices
- Uses authentication (login/password)

**1. Confidentiality**

- Data should not be accessed by unauthorized users
- Achieved using encryption

**2. Integrity**

- Data should not be altered or tampered
- Achieved using hashing and validation

**3. Availability**

- System should be available when needed

- Protected against attacks like DoS

**Challenges in IoT (Internet of Things)**

**1. Security Challenges**

- Devices are vulnerable to hacking and cyberattacks
- Weak authentication and encryption
- Data breaches and privacy issues

👉 Example: Unauthorized access to smart home devices

**2. Privacy Issues**

- IoT devices collect sensitive user data
- Risk of misuse of personal information
- Lack of user awareness and control

**3. Interoperability**

- Different devices use different standards and protocols
- Difficulty in communication between systems

👉 Related standards like OPC UA help solve this, but not fully

**4. Scalability**

- Managing millions of connected devices is complex
- Network congestion and data overload

**5. Data Management**

- Huge volume of data generated
- Storage, processing, and analysis become difficult

**6. Power Consumption**

- Many IoT devices run on batteries
- Limited energy availability reduces device life

**7. Connectivity Issues**

- Unstable or limited network coverage
- Latency and bandwidth problems

**8. Standardization**

- Lack of universal standards

- Different manufacturers follow different approaches

**9. Device Management**

- Monitoring and updating devices remotely is difficult
- Firmware updates and maintenance challenges

**10. Reliability**

- System failures can affect critical applications
- Need for high availability and fault tolerance

**11. Cost**

- High initial setup cost
- Maintenance and infrastructure expenses